

SQL i zapytania SQL SELECT

SQL and SQL SELECT queries

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska

Wydział Matematyki i Nauk Informatycznych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

The role of SQL

- SQL – Structured Query Language (*Strukturalny język zapytań*)– the language used to:
 - Query the data i.e. get raw or aggregated data from one or many tables at the same time
 - Define data structures e.g. create new tables
 - Update data in a database by:
 - Creating new records
 - Modifying records
 - Deleting records
- The SQL language is used by:
 - Software developers as software developers send SQL queries from client applications to read and modify database content
 - Data scientists to retrieve data from a database to use it for further steps of analytical tasks
 - Database administrators to manage database content

SQL revisions

- Different revisions of the standard exist, namely:
 - SQL:2023
 - SQL:2016
 - SQL:2011
 - SQL:2008
 - SQL:2003
 - SQL99 a.k.a. SQL3
 - SQL92 a.k.a. SQL2
 - SQL86 a.k.a. SQL1 developed by ANSI (American National Standards Institute) and ISO (International Standards Organisation)
- As an example, SQL:2023 includes extensions supporting graph-like queries in databases, and improved support for JSON

Standards including 2023 version can be purchased at <https://www.iso.org>

SQL implementations

- SQL standards:
 - Initially based on the common set of features implemented in IBM and Oracle,
 - Only partly implemented by some DBMS vendors,
 - Thus, writing SQL code that is portable among different DBMSs is not easy
 - Different DBMS vendors offer extensions to the language that include procedural statements like PL/SQL by Oracle, Transact-SQL by Microsoft
- Already SQL3 defined:
 - Object-oriented extensions,
 - Core specification that should be implemented by every vendor of RDBMS,
 - Optional packages addressing the needs of data warehouses, spatial data, time-based data, multimedia and other fields.
- Due to the complexity of recent SQL revisions, the division into Core and Optional part is maintained
- Hence, typical SQL dialects are compared against Core SQL as found, e.g. in SQL:2023 Part 2 (Foundation) and Part 11 (Information and definition schemas)

DBMS – compliance with SQL standard – sample compliant subfeatures

Feature ID, Feature	Support
E011, Numeric data types	Oracle fully supports this feature.
E021, Character data types	Oracle fully supports these subfeatures: • E021-01, CHARACTER data type • E021-07, Character concatenation • E021-08, UPPER and LOWER functions • E021-09, TRIM function • E021-10, Implicit casting among character data types

These subfeatures of Core SQL standard are fully supported by Oracle 19c SQL dialect

Source: Oracle 19c SQL Language Reference

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Oracle-Compliance-To-Core-SQL2011.html#GUID-D372D906-805B-49B8-824A-D4697B05B7F8>

DBMS – compliance with SQL standard – sample non-compliant subfeatures

Feature ID, Feature	Support
	Oracle has equivalent functionality for these subfeatures: • E021-04, <code>CHARACTER_LENGTH</code> function: use <code>LENGTH</code> function instead • E021-05, <code>OCTET_LENGTH</code> function: use <code>LENGTHB</code> function instead • E021-06, <code>SUBSTRING</code> function: use <code>SUBSTR</code> function instead • E021-11, <code>POSITION</code> function: use <code>INSTR</code> function instead

These subfeatures of Core SQL standard are handled by equivalent functionality by Oracle 19c SQL dialect

Source: Oracle 19c SQL Language Reference

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Oracle-Compliance-To-Core-SQL2011.html#GUID-D372D906-805B-49B8-824A-D4697B05B7F8>

SQL SELECT

- SELECT statement serves to retrieve data from a table or a set of tables linked by relations (that is by foreign and primary keys)
- SQL SELECT statement can be executed also on the output of another SQL statement – treated as a pseudo-table

SQL SELECT – part I

type	building	floor	room
lecture	Main Building	2	213
lecture	Main Building	3	315
	...		...
lab	Chemistry	4	452

```
SELECT * FROM rooms
```

```
SELECT building, room FROM rooms
```

SELECT - aliases

```
SELECT building AS bld, room as rm  
FROM rooms
```

The output of the query:

bld	rm
Main Building	213
Main Building	315
...	...
Chemistry	452

Lowercase and uppercase names can be used interchangeably in SQL keywords, such as „AS” here.

SELECT – WHERE clause

```
SELECT * FROM rooms where floor>2 and  
type='lecture'
```

The output of the query:

type	building	floor	room
lecture	Main Building	3	315
	...		...

SELECT – IN phrase

```
SELECT * FROM rooms where floor IN (2,4)
```

The output of the query:

type	building	floor	room
lecture	Main Building	2	213
...	...		...
lab	Chemistry	4	452

The IN phrase can be used to specify a set values. Only rows with the column value in this set will be returned.
Note that performance may be reduced when the set includes several values e.g. the output of an embedded SQL SELECT query.

```
SELECT * FROM rooms where floor IN  
(SELECT floor from rooms b where b.type='lecture')
```

SELECT – LIKE phrase

```
SELECT * FROM rooms where building LIKE  
'Main%'
```

The output of the query:

type	building	floor	room
lecture	Main Building	2	213
...	...		...

The LIKE phrase can be used to specify a pattern to be matched by the values in a text column. Examples include:

- % - any number of characters
- _ - any single character
- [b-g] – any character between b and g
- [ghj] – any of the listed characters

Note that patterns should not be used too frequently, due to performance reasons

SELECT – ORDER BY clause

```
SELECT * FROM rooms where floor>=2 ORDER  
BY type,floor
```

The output of the query:

type	building	floor	room
lab	Chemistry	4	452
lecture	Main Building	2	213
lecture	Main Building	3	315
	...		...

SELECT – ORDER BY clause

```
SELECT * FROM rooms [. . .] ORDER BY  
Column_Name [DESC] [,Column_Name. . .]
```

- It is also possible to sort in a descending order
- Ascending order is default
- Please notice that text columns are sorted as text even if they contain numbers thus '1000'<'2'
- When multiple columns are used, rows that have the same values for the first column are compared in terms of the second column and so on

SELECT – DISTINCT clause

```
SELECT DISTINCT type, floor FROM rooms  
where floor>=2
```

The output of the query:

type	floor
lab	4
lecture	2
lecture	3

**Unique combinations of column values/expressions
are obtained**

Linking tables and Cartesian product

Złączanie tabel i iloczyn kartezjański

Rooms

type	Bld_code	Floor	Room
lecture	MAIN	2	213
lab	CHEMISTRY	4	452

Assets

Bld_code	Asset_id	Room	Type
Main	5	213	Chair
Main	7	213	Table

**How to perform cross-table query
e.g. list of assets in main building on
floor 2?**

Linking tables –query styles

WHERE-BASED QUERY

```
SELECT ... FROM  
TABLE1, TABLE2 [, ...]  
WHERE Condition1 [ AND  
Condition2 ...] [...]
```

Example

```
SELECT * FROM rooms,  
assets WHERE  
rooms.bld_code=  
assets.bld_code AND  
rooms.room=assets.room
```

JOIN-BASED QUERY

```
SELECT ... FROM TABLE1  
JOIN TABLE2 ON  
Condition1 [ JOIN ... ON  
...]
```

Example

```
SELECT * FROM rooms  
JOIN assets ON  
rooms.bld_code=  
assets.bld_code AND  
rooms.room=assets.room
```

Query interpretation – step I

Cartesian product is obtained

Bld_code	Floor	Room	Bld_code	Type	Room	...
MAIN	2	213	MAIN	Chair	213	...
CHEM	4	452	MAIN	Chair	213	...
MAIN	2	213	MAIN	Table	213	...
CHEM	4	452	MAIN	Table	213	...

To answer a query referring to multiple tables *Cartesian product* is created first i.e. a temporary listing of all possible combinations of records from the tables is created

Query interpretation – step II

Bld_code	Floor	Room	Bld_code	Type	Room	...
MAIN	2	213	MAIN	Chair	213	

MAIN	2	213	MAIN	Table	213	
------	---	-----	------	-------	-----	--

In the second step Cartesian product is filtered out so as to contain only rows satisfying the condition: `rooms.bld_code=assets.bld_code AND rooms.room=assets.room`

Cross-table queries

- The uniqueness of column names in the result set may be preserved by a DBMS. Thus, a suffix may be added to original column names e.g. `bld_code_a`, `bld_code_b` columns may be observed in the query output
- In case we need to relate to a column in the query:
 - If a column name appears only once in all the involved tables, it is enough to use unqualified name e.g. `floor`
 - Otherwise, fully qualified name should be used e.g. `rooms.bld_code`

Other JOIN categories

Rooms

Bld_code	Floor	Room
MAIN	2	213
CHEM	4	452

Assets

Bld_code	Type	Room	...
MAIN	Chair	213	...
MAIN	Table	213	...

How to obtain room 452 in the chemistry building in the output result set when there are no assets in the room?

Other JOIN categories

Rooms

Bld_code	Floor	Room
MAIN	2	213
CHEM	4	452

Assets

Bld_code	Asset	Room
MAIN	Chair	213
MAIN	Table	213

JOIN-BASED QUERY

```
SELECT ... FROM TABLE1  
[INNER|LEFT|RIGHT|FULL]  
JOIN TABLE2 ON  
Condition1[ AND  
Condition2 ...] [...]  
[ JOIN ...]
```

Other JOIN categories

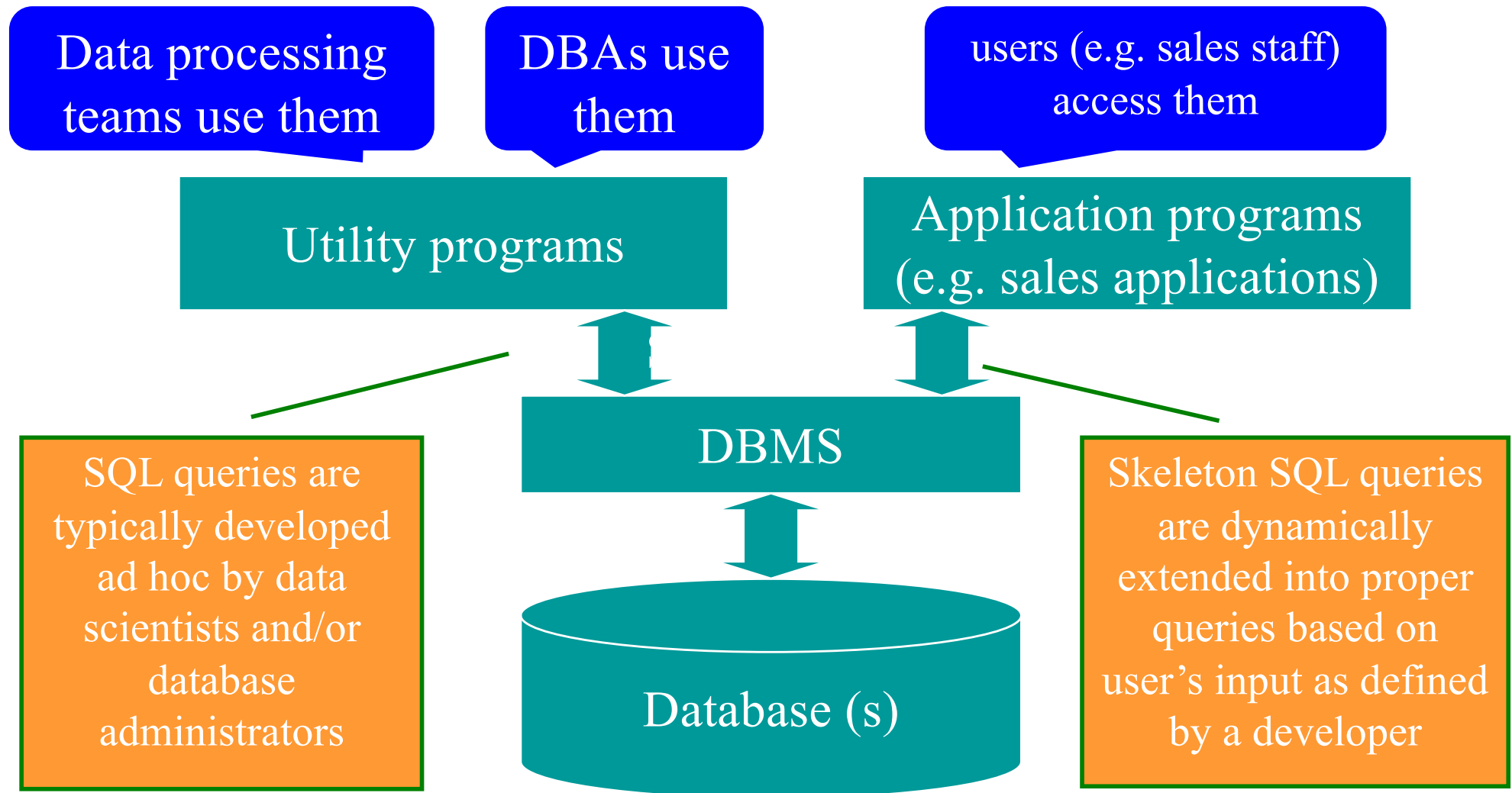
Table_1

Table_2

LEFT JOIN					..			
INNER JOIN			...	...	...			
		...	...	..				
								...
								
								

FULL JOIN will include all the combinations of records i.e. a sum of LEFT and RIGHT approach.

DBMS and SQL



SQL and analytical software

RStudio, SAS, Business Intelligence,

Reporting/analytical tools

Two categories of applications can be identified:

- Tools making it possible to develop an ad hoc SQL query and submit it e.g. R and RStudio
- Tools, which aim to automatically develop SQL queries based on user's input (BI software)

DBMS

Database (s)

Summary

- SQL is a key language in databases domain
- The knowledge of SQL is fundamental for:
 - Software developers
 - Data scientists
 - Database administrators
- In spite of standardisation efforts, different SQL dialects exist
- While most of SQL knowledge can be transferred between DBMSs, SQL queries are typically not portable due to dialect differences

Pytania ?

Zapraszam w trakcie wykładu
i w trakcie konsultacji

Miejsce i termin konsultacji
- na stronie:

[https://pages.mini.pw.edu.pl/
~grzendam/pl/dydaktyka.html](https://pages.mini.pw.edu.pl/~grzendam/pl/dydaktyka.html)



Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.